

Impacto de la disponibilidad de ChatGPT en el desarrollo de software orientado a Ciencia de Datos (2020–2023)

Sustentación de tesis para optar al título de Licenciado en Economía

Jesus Alberto Nicho Rosado
Karl Willem Janampa Aparicio

Pontificia Universidad Católica del Perú
Facultad de Ciencias Sociales
Asesor: Alexander Wilder Quispe Rojas

Lima, 2026

Agenda de la sustentación

1. Motivación y problemática
2. Pregunta e hipótesis
3. Marco teórico
4. Evidencia empírica previa
5. Línea de tiempo de ChatGPT
6. Datos y diseño del panel
7. Metodología (DiD, CS, SDiD)
8. Resultados principales
9. Pruebas de robustez
10. Limitaciones y conclusiones

1. Motivación

- ChatGPT (OpenAI, 30 noviembre 2022) marcó un punto de inflexión en la asistencia con IA generativa para tareas de programación.
- Su escala fue cualitativamente distinta a tecnologías previas (Copilot, Codex): **servicio gratuito, masivo y conversacional**.
- Su **disponibilidad geográfica heterogénea** —bloqueado en China, Rusia, Irán, Cuba, Corea del Norte, entre otros— generó un escenario cuasi-experimental natural.
- El sector software se proyectaba a USD 600 000 millones para 2025 (Statista, 2023). Toda mejora marginal en productividad tiene consecuencias agregadas relevantes.

¿Por qué importa?

La pregunta práctica

Si ChatGPT efectivamente aumenta la productividad o la entrada de desarrolladores, sus implicaciones cruzan tres dimensiones:

Política pública

Subsidios al acceso, regulación de servicios IA, restricciones internacionales.

Empresas

Inversión en suscripciones premium, programas de adopción, evaluación de productividad.

Educación

Currículos de ciencia de datos, formación de juniors, certificaciones laborales.

Sin evidencia causal robusta, las decisiones se toman a ciegas.

2. Pergunta e hipótesis

Pregunta central

¿La **disponibilidad de ChatGPT** causó un incremento del número de desarrolladores activos en GitHub en los países con acceso, comparado con los países donde el servicio fue restringido, entre 2020 y 2023?

Unidad de análisis: país $\times$ lenguaje de programación $\times$ trimestre.

Tratamiento: acceso público a ChatGPT a partir de noviembre 2022.

Variable de resultado: *unique pushers* por cada 100 000 habitantes.

Período: Q1-2020 a Q4-2023 (16 trimestres).

- **H1.** La accesibilidad a ChatGPT se asocia positivamente con el desarrollo de software en los países tratados.
- **H2.** El efecto es **heterogéneo por lenguaje**: más fuerte en Python y JavaScript dada su vasta documentación técnica disponible en internet, recurso fundamental para el entrenamiento del modelo.

Mecanismos teóricos detrás de las hipótesis

- 1 **Reasignación de tareas** (Acemoglu & Restrepo, 2018, 2019): la IA libera tareas rutinarias, expande la frontera de proyectos rentables y atrae programadores marginales.
- 2 **Reducción de costos de búsqueda y aprendizaje** (Stigler, 1961): ChatGPT colapsa el costo de explorar documentación y foros a niveles cercanos a cero.

3. Marco teórico

Tecnología como reasignador de tareas

El cambio tecnológico actúa en dos canales simultáneos sobre el trabajo:

Displacement effect

ChatGPT automatiza tareas previamente del programador:

- Búsqueda de sintaxis
- Redacción de *boilerplate*
- Depuración rutinaria

Reinstatement effect

Al bajar el costo marginal de producir código:

- Proyectos antes inviables se vuelven rentables.
- Programadores marginales pueden entrar.
- La frontera productiva se expande.

Efecto neto sobre la actividad de desarrollo: positivo bajo condiciones plausibles.

Costos de búsqueda y de aprendizaje

Stigler (1961) modela la búsqueda como un costo que el agente paga para reducir incertidumbre sobre alternativas.

En desarrollo de software, este costo se traduce en:

- Tiempo dedicado a leer documentación oficial.
- Búsquedas en Stack Overflow, foros, tutoriales.
- Experimentación con código que falla.

ChatGPT actúa como un *shock* negativo sobre estos costos: ofrece respuestas contextualizadas e instantáneas.

Evidencia coherente: Del Río-Chanona et al. (2023)

–16 % en actividad de Stack Overflow tras el lanzamiento de ChatGPT $\Rightarrow$ reorganización endógena de los canales de búsqueda.

4. Evidencia empírica

Evidencia empírica: impacto económico de la IA

Autor	Efecto	Resultado breve
Brynjolfsson et al. (2023)	↑ Positivo	↑ 14 % la productividad en atención al cliente, especialmente en empleados con menor experiencia.
Gallea (2023)	↑ Positivo	↓ preguntas comunes en Stack Overflow y orienta a los usuarios hacia consultas más complejas.
Wu et al. (2024)	↑ Positivo	↑ valor de mercado de las empresas chinas que decidieron integrar la IA.
Liu et al. (2023)	≈ Neutro	↓ la demanda de trabajos automatizables, pero favoreció a freelancers que lo integraron en su servicio.
Del Río-Chanona et al. (2023)	↓ Negativo	↓ uso de foros como Stack Overflow, afectando la generación de datos públicos para entrenar IA.
Demirci et al. (2023)	↓ Negativo	↓ demanda de tareas automatizables y de generación de imágenes.
Hui et al. (2023)	↓ Negativo	↓ el empleo y los ingresos de <i>freelancers</i> en ocupaciones vulnerables como redacción y edición.

Evidencia empírica: capacidades por lenguaje de programación

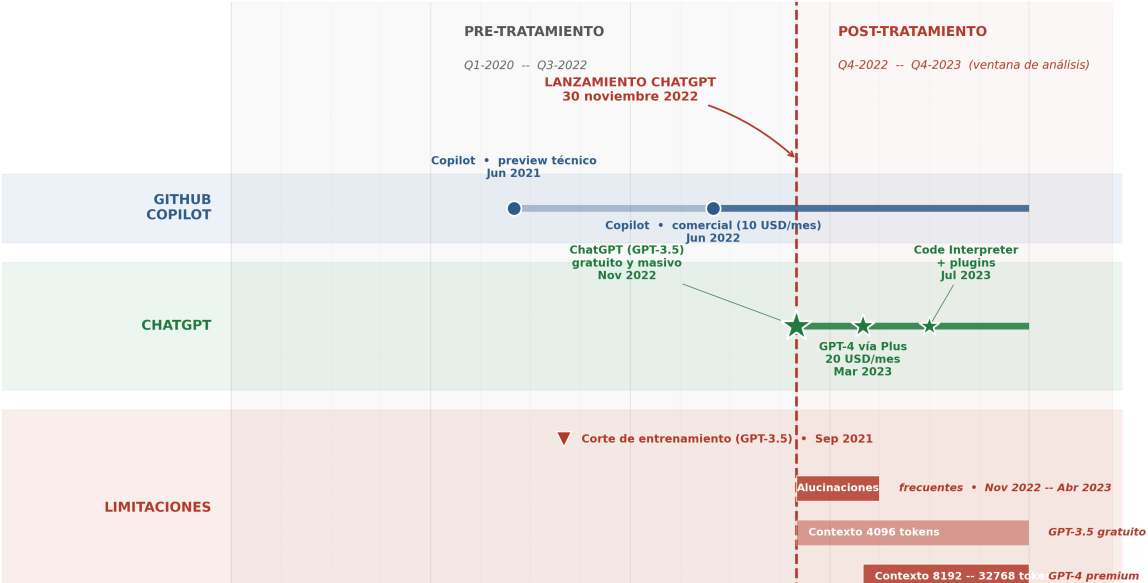
Autor	Efecto	Resultado breve
Buscemi (2023)	↑ Positivo	En ciencia de datos, Julia mostró el mejor desempeño (81,5 %), seguido de R y Python, destacando la eficacia del modelo en lenguajes populares y especializados en análisis estadístico.
Chai et al. (2024)	↑ Positivo	Usando el <i>benchmark</i> MCEVAL, se evidenció que el rendimiento de ChatGPT-3.5 Turbo varía ampliamente entre lenguajes: destaca en Python, Java y Ruby, pero muestra limitaciones en lenguajes menos comunes como Lisp, Scheme y VimL.
Li y Krishnamachari (2024)	↑ Positivo	MultiPL-LeetCode mostró que el rendimiento de ChatGPT-3.5 Turbo varía según el lenguaje: es alto en Python, Java y C++, pero deficiente en lenguajes funcionales como Elixir, Erlang y Racket.

Brecha en la literatura

Ningún estudio analiza específicamente el impacto causal de ChatGPT sobre el **desarrollo de software** desagregado por lenguaje a nivel de país.

5. Línea de tiempo

Línea de tiempo: ChatGPT y limitaciones (2020–2023)



La ventana postratamiento (Q4-2022 a Q4-2023) no fue homogénea:

Q4-2022 – Q1 2023

- GPT-3.5 gratuito
- Alucinaciones frecuentes
- Contexto 4 096 tokens
- Corte de entrenamiento: Sep 2021

Q2-Q4 2023

- GPT-4 vía Plus (20 USD/mes)
- Reducción de alucinaciones
- Contexto 8 192–32 768 tokens
- Code Interpreter, *plugins*

Esto abre la posibilidad de un efecto no monotónico en el corto plazo.

6. Datos

Fuente: GitHub Innovation Graph

Base pública mantenida por GitHub con métricas agregadas de actividad de desarrollo por país, lenguaje y trimestre.

Variable de resultado: *unique pushers* por cada 100 000 habitantes.

Unique pushers = cantidad de desarrolladores únicos que al menos hicieron un `git push` en GitHub durante un trimestre, normalizado por población.

- **Mide:** entrada/persistencia de programadores activos.
- **No mide:** intensidad por persona, calidad del código, valor agregado.
- **Limitación reconocida:** países como China usan plataformas alternativas (Gitee) $\Rightarrow$ posible subestimación en el control.

Dimensiones del panel:

- **159 países** (N): 130 tratados + 29 control.
- **10 lenguajes**: C, C#, C++, Go, Java, JavaScript, PHP, Python, Ruby, TypeScript.
- **16 trimestres**: Q1-2020 a Q4-2023.
- **Total**: 25 440 observaciones.

Período de pretratamiento: 12 trimestres (Q1-2020 a Q3-2022).

Período de tratamiento: 4 trimestres (Q4-2022 a Q4-2023).

Exclusiones: EU (agregado), Kosovo (sin código ISO procesable), Hong Kong (atípico).

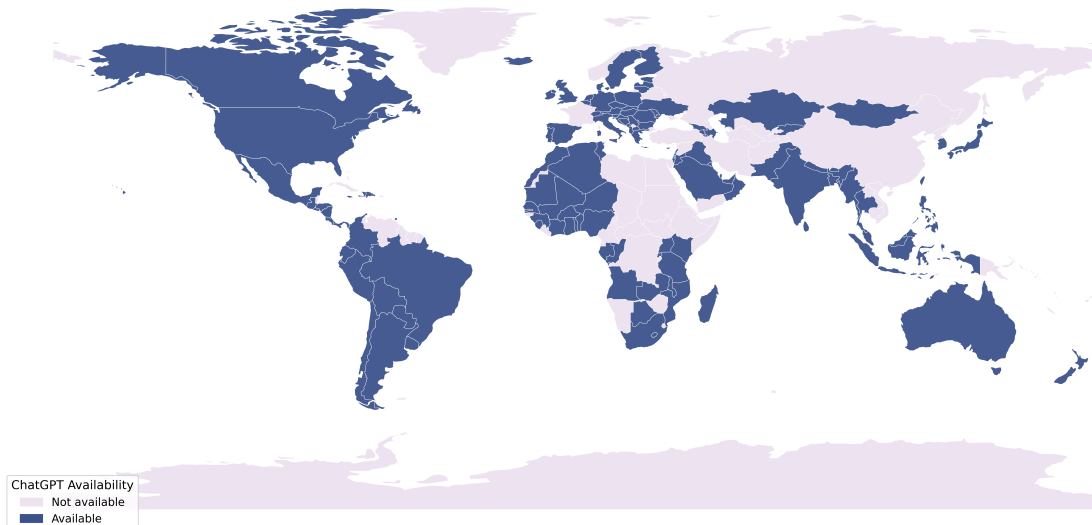
Países sin acceso a ChatGPT (control)

Censores severos: China, Cuba, Irán, Bielorrusia, Rusia, Siria.

Otros: Afganistán, Bahréin, Burundi, Camboya, Camerún, Congo (RD), Egipto, Eswatini, Etiopía, Guadalupe, Macao, Puerto Rico, Reunión, Somalia, Sudán, Tayikistán, Turkmenistán, Turquía, Uzbekistán, Venezuela, Vietnam, Yemen, Zimbabue, Libia, Laos.

Mapa global de disponibilidad de ChatGPT

Global ChatGPT Availability

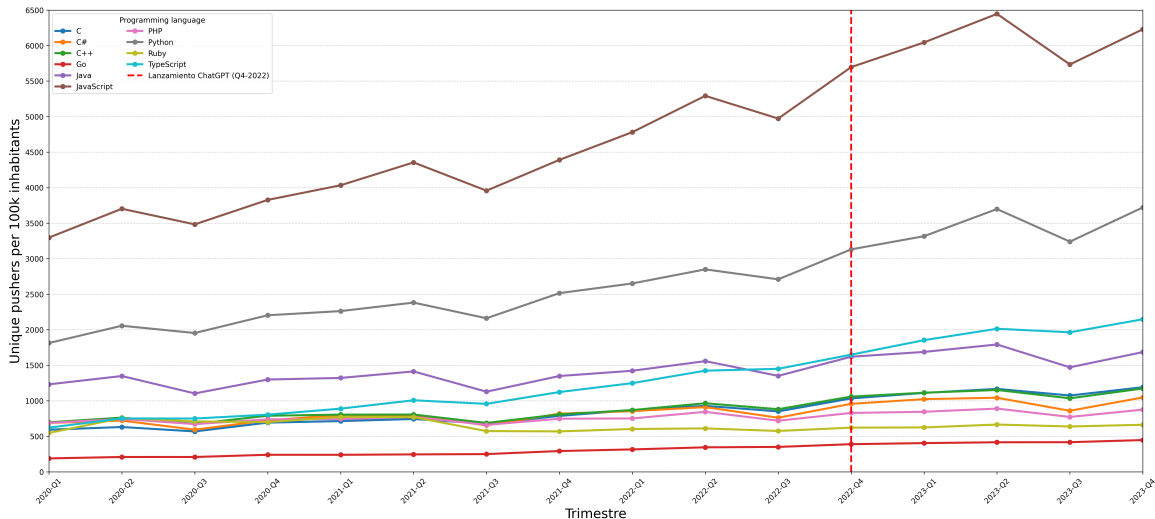


Media de *unique pushers* por 100k habitantes

Lenguaje	Pretratamiento		Postratamiento	
	Control	Tratado	Control	Tratado
JavaScript	18,84	51,00	34,29	58,98
Python	13,30	22,71	22,21	33,46
TypeScript	3,46	17,32	11,79	18,83
Java	6,59	13,58	10,81	16,14
C	4,88	7,16	8,73	10,91
C++	5,28	7,77	8,84	10,82
C#	3,01	7,39	4,41	9,64
Go	1,74	3,63	4,85	4,05
Ruby	3,13	6,07	4,00	6,28
PHP	2,27	7,54	3,46	8,24

Lectura: ambos grupos crecen en el período. El efecto causal requiere comparar las **tasas** de cambio, no los niveles.

Tendencias temporales (países con ChatGPT)



Línea vertical roja: lanzamiento de ChatGPT (Q4-2022). **Atención a la aceleración de JavaScript, Python y TypeScript post-tratamiento.**

7. Metodología

Marco común: resultados potenciales

Para cada unidad i en el período t :

$$Y_{it} = Y_{it}^{(0)} + \tau_{it} W_{it}$$

donde $W_{it} = \mathbf{1}[i \in \mathcal{T}, t \geq T_0 + 1]$.

Parámetro de interés: efecto promedio del tratamiento sobre los tratados (ATT):

$$\tau = \frac{1}{N_{tr} T_{post}} \sum_{i \in \mathcal{T}} \sum_{t > T_0} \tau_{it}$$

Tres estrategias para estimar τ

- 1 **DiD** — Diferencias en Diferencias (Card & Krueger, 1994).
- 2 **CS** — Control Sintético (Abadie, Diamond & Hainmueller, 2010).
- 3 **SDiD** — Diferencias en Diferencias Sintética (Arkhangelsky et al., 2021). **Método principal.**

Especificación de efectos fijos de dos vías:

$$Y_{it} = \mu + \alpha_i + \beta_t + \tau^{DiD} W_{it} + \varepsilon_{it}$$

Estimador explícito:

$$\hat{\tau}^{DiD} = (\bar{Y}_{post, \mathcal{T}} - \bar{Y}_{pre, \mathcal{T}}) - (\bar{Y}_{post, \mathcal{C}} - \bar{Y}_{pre, \mathcal{C}})$$

Supuesto crítico: tendencias paralelas pre-tratamiento.

Debilidad para este estudio: el control es heterogéneo (países autoritarios) y las tendencias pretratamiento podrían no ser paralelas.

Método 2: Control Sintético (CS)

Estimador:

$$\hat{\tau}^{SC} = \frac{1}{N_{tr} T_{post}} \sum_{i \in \mathcal{T}} \sum_{t > T_0} \left(Y_{it} - \sum_{j \in \mathcal{C}} \hat{\omega}_j^{SC} Y_{jt} \right)$$

Pesos $\hat{\omega}_j^{SC}$:

$$\hat{\omega}^{SC} = \arg \min_{\omega} \sum_{t=1}^{T_0} \left(\sum_{j \in \mathcal{C}} \omega_j Y_{jt} - \bar{Y}_t^{\mathcal{T}} \right)^2 \quad \text{s.a.} \quad \omega_j \geq 0, \sum_j \omega_j = 1$$

Idea: construir un “tratado sintético” como promedio ponderado de los controles, que copie la trayectoria pretratamiento del tratado real.

Ventaja: maneja heterogeneidad en el grupo de control.

Limitación: mayor varianza cuando el tratado es agregado de muchas unidades heterogéneas.

Método 3: SDiD (Arkhangelsky et al., 2021)

Combina DiD y CS asignando pesos a unidades y a períodos:

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \arg \min \sum_{i,t} \hat{\omega}_i^{sdid} \hat{\lambda}_t^{sdid} (Y_{it} - \mu - \alpha_i - \beta_t - W_{it} \tau)^2$$

Pesos unitarios $\hat{\omega}^{sdid}$: resuelven un problema CS con regularización.

Pesos temporales $\hat{\lambda}^{sdid}$: dan más peso a períodos pretratamiento cercanos a T_0 .

Por qué SDiD es el principal

- Asintóticamente normal con muchas unidades tratadas ($N_{tr} = 130$).
- Robusto a violaciones del supuesto de tendencias paralelas.
- Combina la flexibilidad de CS con la simplicidad de DiD.

Inferencia: *bootstrap* por remuestreo de unidades.

- DiD y SDiD: 100 replicaciones.
- CS: 2 000 replicaciones (mayor varianza por iteración).

Pruebas de robustez adicionales:

- 1 **Cuadro 4:** con variables de control (uso de PC y de internet, Banco Mundial).
- 2 **Cuadro 5:** grupo de control restringido (excluye los 6 censores severos de Freedom House).
- 3 **Cuadro 6:** placebos in-space + sensibilidad LOO al pool de donantes (Abadie, 2010, 2021).

8. Resultados

Resultados principales: ATT por método (Cuadro 3)

Lenguaje	DiD	CS	SDiD
JavaScript	13,635*** (2,719)	9,451* (5,307)	8,303*** (3,180)
Python	7,688*** (1,193)	1,604 (3,552)	4,525*** (1,321)
TypeScript	7,181*** (0,832)	3,263 (2,290)	3,078*** (0,873)
Java	2,810*** (0,827)	2,206 (2,089)	1,725** (0,872)
C	2,689*** (0,365)	1,634 (1,137)	1,276*** (0,451)
C++	2,210*** (0,417)	1,212 (0,959)	1,131*** (0,435)
C#	1,499*** (0,381)	1,001 (1,263)	0,443 (0,379)
Go	1,411*** (0,177)	0,686 (0,606)	0,720*** (0,269)
PHP	0,904* (0,507)	1,078 (1,013)	0,916** (0,465)
Ruby	0,262 (0,266)	1,521 (0,946)	0,808*** (0,274)

Errores estándar *bootstrap* entre paréntesis. * $p < 0,10$ ** $p < 0,05$ *** $p < 0,01$.

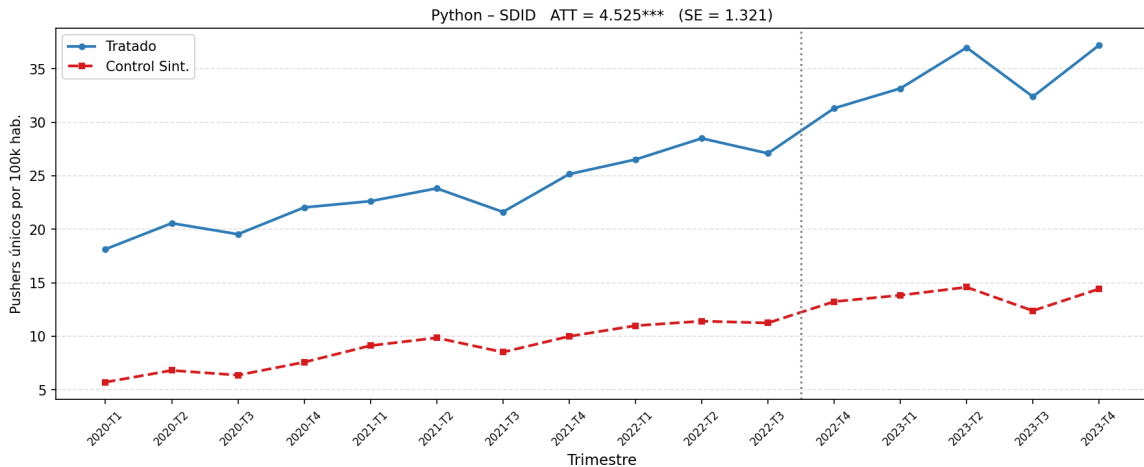
Hallazgos clave del Cuadro 3

- **Nueve de diez lenguajes** muestran ATT positivo y significativo en el SDiD (1 % o 5 %).
- El único lenguaje sin efecto: **C#** (SDiD = 0,443, no significativo).
- Magnitudes más grandes: **JavaScript** (+8,3), **Python** (+4,5), **TypeScript** (+3,1).
- Lenguajes con mayor representación en la web y data science presentan mayores efectos $\Rightarrow$ consistente con la **H2**.

Pero hay que ser cuidadosos

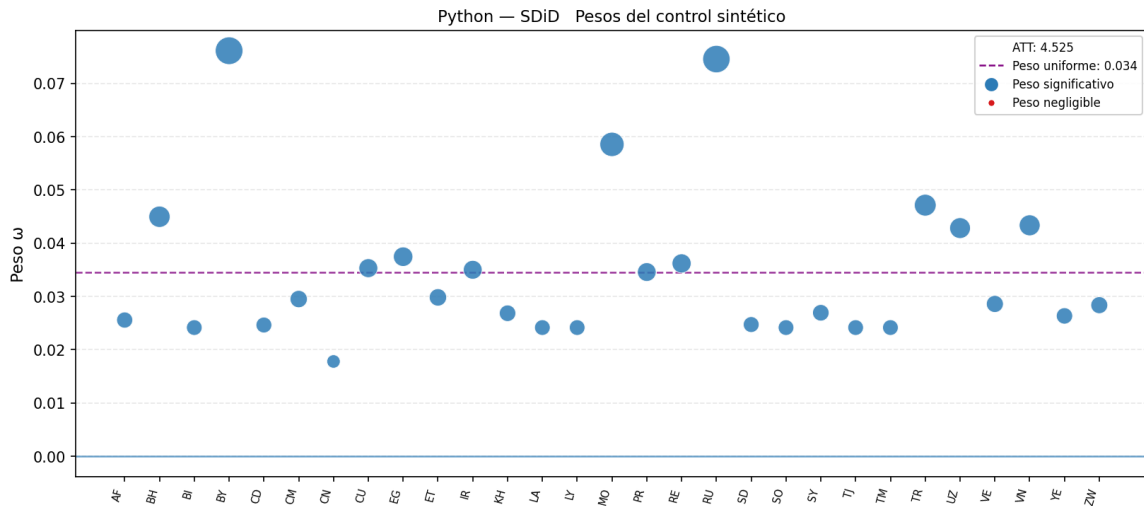
La significancia *bootstrap* no es la única evidencia que conviene revisar. Las pruebas de placebo refinan la lectura $\Rightarrow$ siguiente sección.

Tendencias y contrafactual: Python



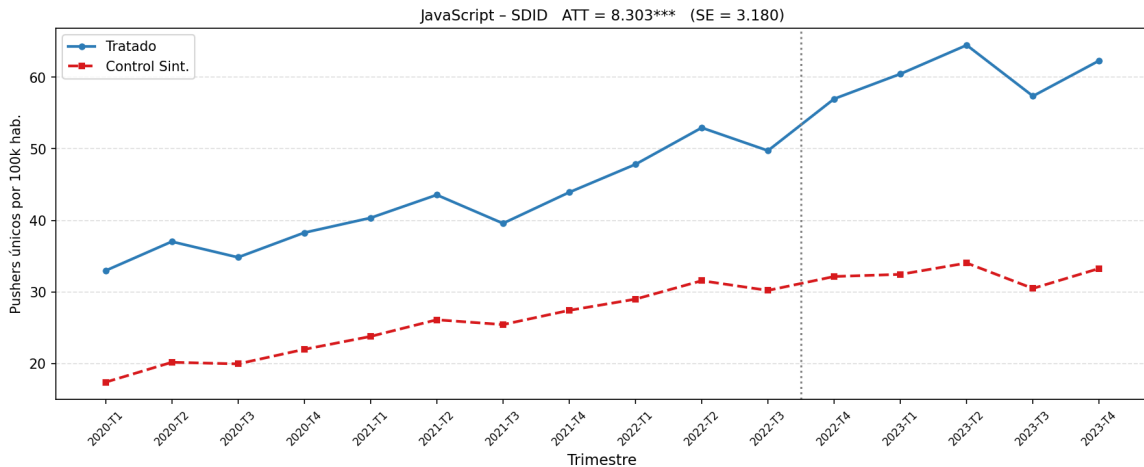
Tratado (azul) vs. control sintético SDiD (rojo). Las trayectorias coinciden hasta Q4-2022; luego divergen. **ATT** = 4,525 unique pushers por 100k.

Pesos del control sintético: Python



Puntos azules: donantes con peso significativo ($\geq 30\%$ del peso uniforme). **Puntos rojos:** pesos negligibles. Línea morada: peso uniforme $1/N_c \approx 0,034$.

Tendencias y contrafactual: JavaScript



Atención: ligera pre-tendencia ascendente antes de Q4-2022 $\Rightarrow$ el efecto causal en JavaScript debe interpretarse con cautela pese a la alta significancia.

9. Robustez

Cuadro 5: se excluyen los 6 censores severos (CN, CU, IR, BY, RU, SY).

Lenguaje	SDiD completo	SDiD restringido
JavaScript	8,303***	8,143***
Python	4,525***	4,378***
TypeScript	3,078***	3,025***
Java	1,725**	1,734**
C	1,276***	1,235***
C++	1,131***	1,094***
PHP	0,916**	0,893**
Ruby	0,808***	0,781***
Go	0,720***	0,701***
C#	0,443	0,398

Los resultados son cualitativa y cuantitativamente similares.

Cuadro 6: dos pruebas no paramétricas por lenguaje.

Lenguaje	ATT obs.	Media plac.	p-valor plac.	LOO mín	LOO máx
JavaScript	8,303	−0,161	0,069	4,642	9,322
Python	4,525	0,355	0,069	3,553	5,111
TypeScript	3,078	0,022	0,069	1,904	3,598
Java	1,725	−0,065	0,103	1,072	2,646
C	1,276	0,039	0,103	1,095	1,436
C++	1,131	0,071	0,138	1,016	1,278
PHP	0,916	−0,243	0,172	0,415	1,124
Ruby	0,808	−0,009	0,034	0,667	0,853
Go	0,720	−0,007	0,034	0,587	0,839
C#	0,443	−0,010	0,207	0,411	0,824

Placebos in-space: 29 placebos por lenguaje. LOO: 29 reestimaciones eliminando un control a la vez.

Tres niveles de evidencia

Fuerte ($p \leq 0,05$)

- Go ($p = 0,034$)
- Ruby ($p = 0,034$)

ATT atípico vs. placebos.

Moderada ($p \leq 0,10$)

- JavaScript (0,069)
- Python (0,069)
- TypeScript (0,069)

Lenguajes priorizados por la hipótesis.

Débil ($p > 0,10$)

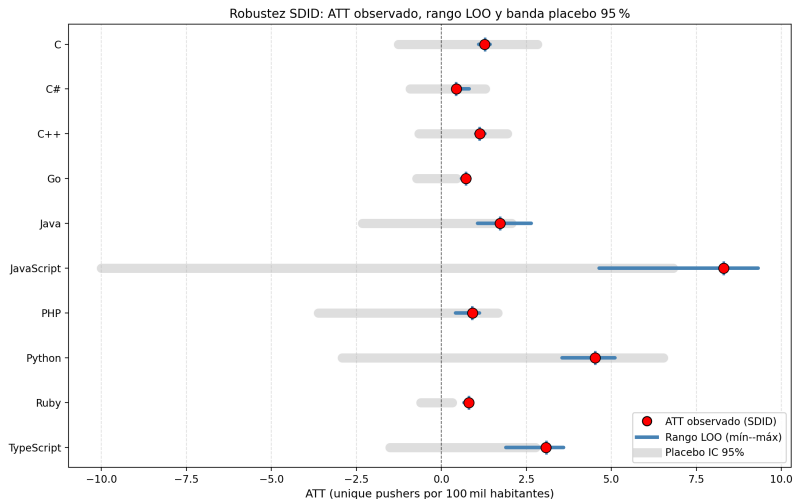
- C, C++, Java
- PHP
- C#

Cautela en interpretación causal.

Lectura honesta

La significancia *bootstrap* para C, C++ y Java coexiste con p-valores placebo $> 0,10$. Esto sugiere heterogeneidad estructural no completamente absorbida por el contrafactual.

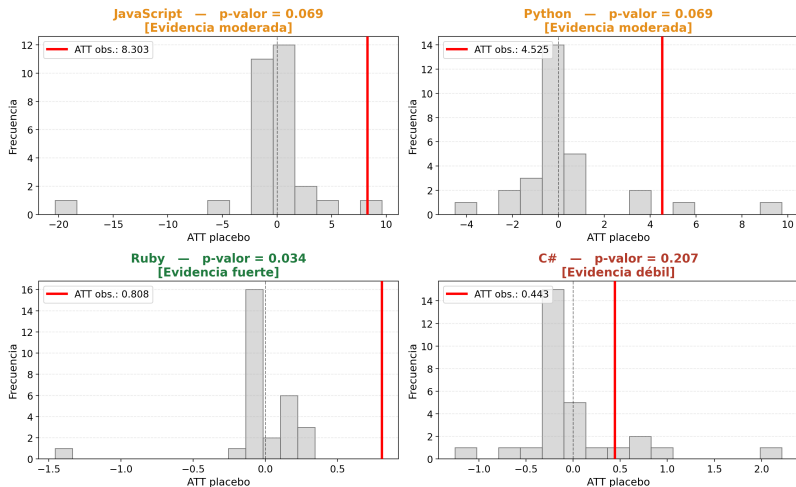
Forest plot: visualización conjunta



Punto rojo: ATT observado. **Barra azul:** rango LOO. **Banda gris:** intervalo placebo 95 %.

Distribución placebo: 4 lenguajes representativos

Distribución de ATT placebo — 4 lenguajes representativos



Línea roja: ATT observado. Histograma gris: 29 ATT placebo construidos en el grupo de control.

10. Limitaciones y conclusiones

- 1 **Asignación no aleatoria al control.** Los países sin ChatGPT son geopolíticamente distintos. El Cuadro 5 mitiga al excluir censores severos, pero no elimina el sesgo.
- 2 **Interferencia por VPN.** Una fracción de desarrolladores en países restringidos pudo haber accedido vía VPN. Esto **atenúa** el efecto estimado hacia cero.
- 3 **Ventana postratamiento corta (4 trimestres).** Limita inferencias de largo plazo y exige cautela frente a tendencias generales contemporáneas (Bard, Copilot Chat, Claude).
- 4 **Outcome limitado.** *Unique pushers* mide cantidad, no calidad ni intensidad. Países como China usan Gitee $\Rightarrow$ posible subestimación adicional en el control.
- 5 **Pre-tendencias en JavaScript.** El análisis de evento revela ligera pre-tendencia $\Rightarrow$ interpretación causal con cautela.

Conclusiones (1): hallazgo central

Hallazgo

La disponibilidad de ChatGPT incrementó la actividad de desarrollo de software en los países con acceso, con **efecto positivo y robusto** en los lenguajes priorizados por la hipótesis teórica: JavaScript, Python y TypeScript.

Apoyo teórico:

- Consistente con **Acemoglu & Restrepo (2018)**: reasignación de tareas hacia segmentos con menor productividad pre-tecnológica.
- Consistente con **Stigler (1961)**: *shock* negativo sobre costos de búsqueda.

Validación empírica:

- Tres métodos econométricos (DiD, CS, SDiD) coinciden en el signo para lenguajes principales.
- Pruebas de placebo y LOO sostienen la interpretación causal en el subconjunto robusto.

Conclusiones (2): heterogeneidad

Lenguajes	Evidencia	Interpretación
Go, Ruby	Fuerte	Efecto causal claramente identificado
JavaScript, Python, TypeScript	Moderada	Lenguajes priorizados por H2; efecto plausible
C, C++, Java	Débil	Significancia <i>bootstrap</i> pero no en placebos
PHP, C#	Débil	Efectos pequeños o ausentes

Las recomendaciones de política se fundamentan en el subconjunto con evidencia fuerte o moderada.

Recomendación 1: Acceso subsidiado para desarrolladores junior

Incentivos a empresas tecnológicas para financiar suscripciones **premium** (ChatGPT Plus) por períodos iniciales de **6 meses renovables** a desarrolladores junior orientados a ciencia de datos.

Recomendación 2: Bonos por productividad asistida por IA

Sistema de **bonos económicos** a trabajadores que evidencien mejoras de rendimiento atribuibles al uso de herramientas de IA, con seguimiento bimensual y estándares de QA.

Ambas medidas son consistentes con el mecanismo identificado por Acemoglu & Restrepo (2018): la IA expande la frontera productiva especialmente entre los segmentos del mercado laboral con menor productividad pre-tecnológica.

Contribución de esta tesis:

- Primer estudio que estima el efecto causal de ChatGPT sobre la **actividad de desarrollo de software** desagregada por lenguaje de programación.
- Aplicación del estimador SDiD (Arkhangelsky et al., 2021) en un contexto con $N_{tr} = 130$ unidades tratadas.
- Set completo de pruebas de robustez: placebos in-space + LOO donor pool + grupo de control restringido.

Líneas de investigación futuras:

- 1 Ventana postratamiento extendida hasta 2025-2026.
- 2 Análisis formal de mecanismos: correlación entre efecto y representación del lenguaje en datos de entrenamiento.
- 3 Heterogeneidad por nivel de ingreso, infraestructura digital y composición educativa.
- 4 Outcomes complementarios: commits, líneas de código, repositorios creados.

Gracias.

Quedo atento a sus consultas y comentarios.

Jesus Alberto Nicho Rosado • Karl Willem Janampa Aparicio

Asesor: Alexander Wilder Quispe Rojas

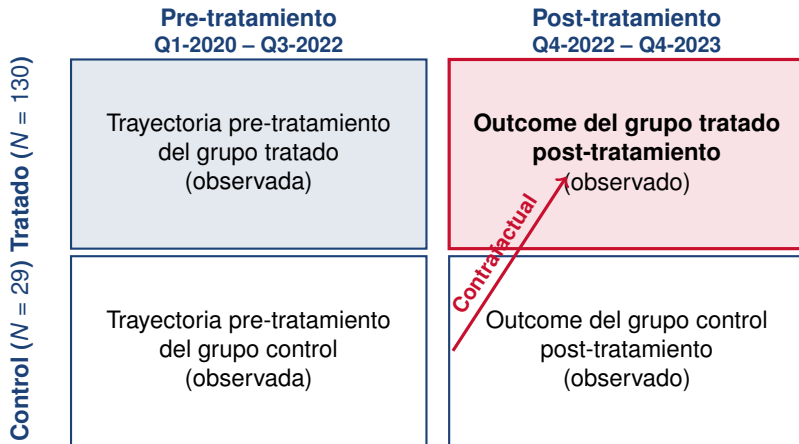
Pontificia Universidad Católica del Perú • Lima, 2026

Apéndice

Slides de respaldo para preguntas

- A.1 Diseño cuasi-experimental (matriz 2×2)
- A.2 Cuadro 4 — Resultados con variables de control
- A.3 Event study SDID — Python (pre-tendencia plana)
- A.4 Event study SDID — TypeScript (pre-tendencia plana)
- A.5 Event study SDID — JavaScript (contraste, pre-tendencia visible)
- A.6 Pesos del control sintético — JavaScript

A.1 Diseño cuasi-experimental (matriz 2×2)



$$ATT = (\text{Tratado}_{\text{post}} - \text{Tratado}_{\text{pre}}) - (\text{Control}_{\text{post}} - \text{Control}_{\text{pre}}).$$

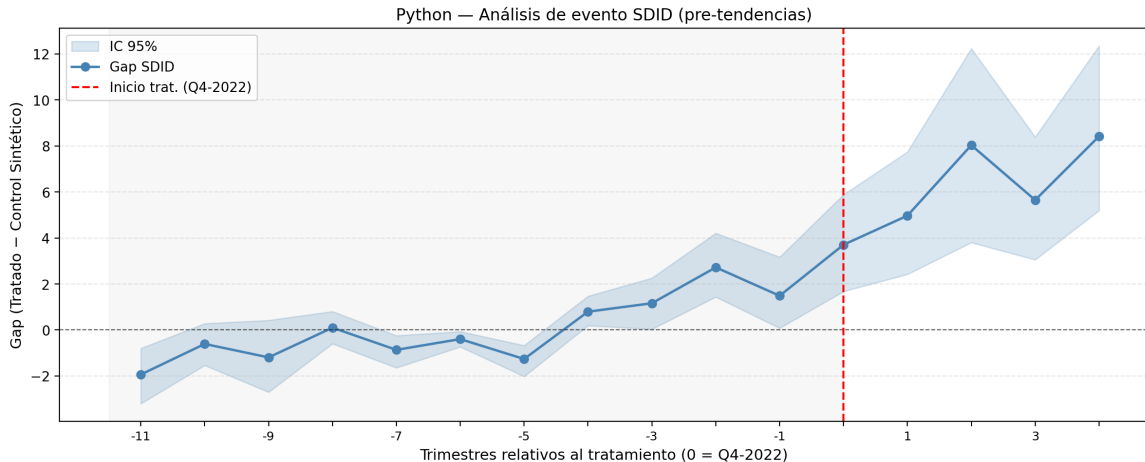
El SDID pondera las celdas para construir un contrafactual más preciso que el DiD clásico.

A.2 Resultados con variables de control (Cuadro 4)

Lenguaje	DiD	CS	SDiD
JavaScript	14,106*** (2,856)	10,470** (4,738)	9,142*** (3,228)
Python	7,831*** (1,147)	2,028 (3,531)	4,700*** (1,324)
TypeScript	7,189*** (0,846)	2,742 (2,244)	3,058*** (0,870)
Java	2,831*** (0,832)	2,224 (2,038)	1,726** (0,874)
C	2,774*** (0,367)	1,433 (1,240)	1,396*** (0,424)
C++	2,236*** (0,419)	1,242 (0,979)	1,157*** (0,437)
C#	1,475*** (0,383)	1,049 (1,252)	0,422 (0,386)
Go	1,411*** (0,176)	0,689 (0,606)	0,728*** (0,270)
PHP	0,994* (0,523)	0,974 (1,029)	1,034** (0,470)
Ruby	0,312 (0,272)	1,605* (0,855)	0,864*** (0,278)

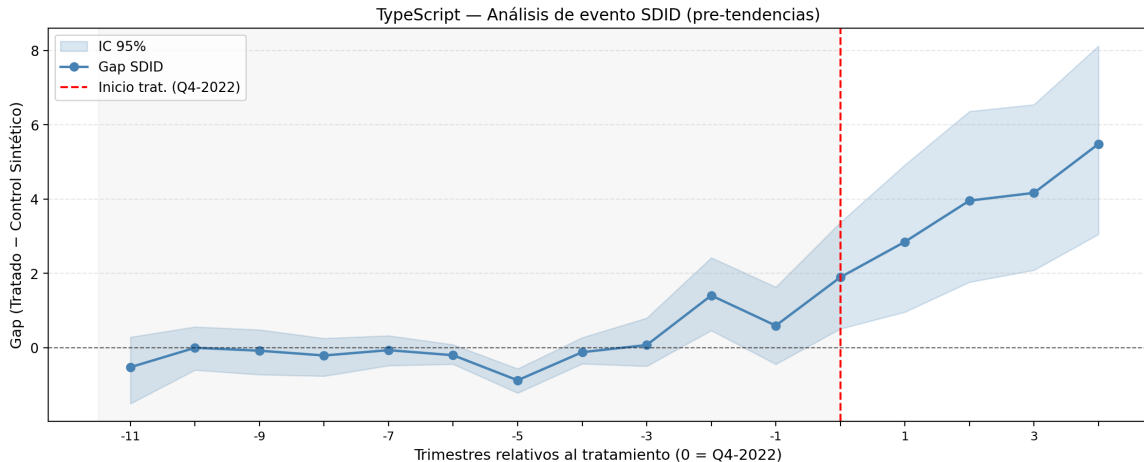
Controles: % individuos que usan computadora y % que usan internet (Banco Mundial). **Los efectos se mantienen e incluso se fortalecen.**

A.3 Event study SDID — Python



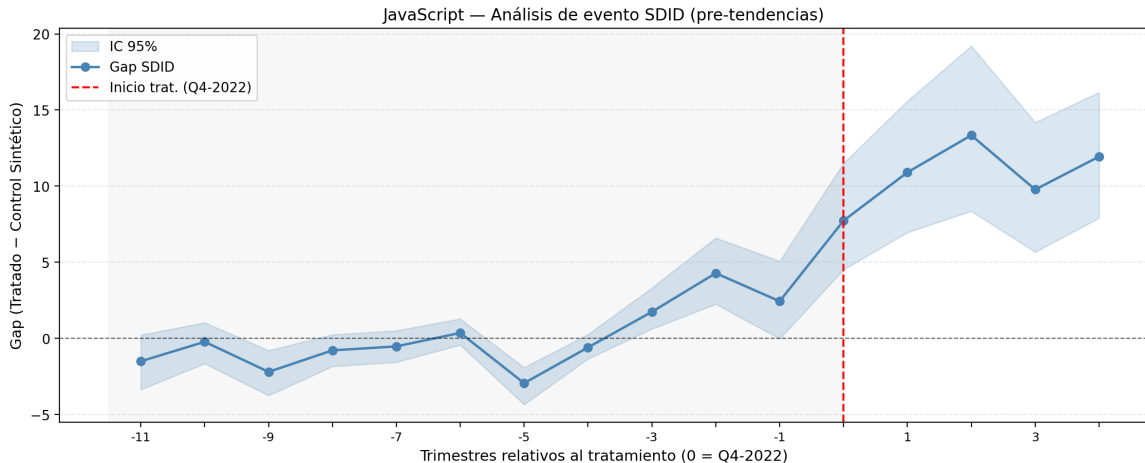
Pre-tendencia plana (gap cerca de cero antes de Q4-2022): el supuesto de paralelismo se cumple para Python $\Rightarrow$ el SDID estima el efecto causal de manera consistente.

A.4 Event study SDID — TypeScript



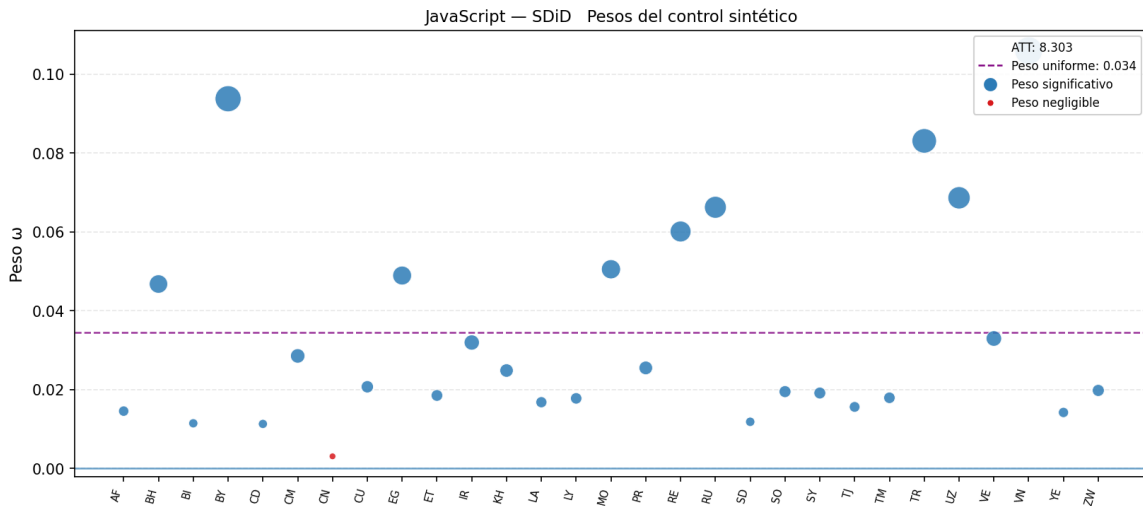
Pre-tendencia plana también para TypeScript. El efecto post-tratamiento es claro y la divergencia ocurre a partir de Q4-2022.

A.5 Event study SDID — JavaScript (contraste)



Atención: en JavaScript se observa una ligera pre-tendencia creciente. El problema es **específico de este lenguaje**, no del método (ver A.3 y A.4 con pre-tendencias planas).

A.6 Pesos del control sintético — JavaScript



Puntos azules: donantes con peso significativo ($\geq 30\%$ del peso uniforme). El contrafactual de JavaScript se construye con un conjunto diversificado de donantes, lo que reduce dependencia de cualquier país individual.